SIMATS ENGINEERING

ASSESSMENT TOOL 1 — CASE STUDY

Course Name: Object Oriented Analysis and Design

Course Code: CSA1105

Course Outcome: CO3 — Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models (BL4)

Weightage: 40%

Questions Answered: 2 of 6 — Smart Hospital Management System, Smart Banking System | **Total Marks: 40**

Code of Conduct:

I _____RASHMI NAURENE(192521357)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____Rashmi Naurene_____

Case 1. Smart Hospital Management System [20 Marks]

A multi-specialty hospital plans to implement a Smart Hospital Management System that enables patients to book appointments, doctors to maintain electronic medical records, laboratories to upload test reports, and pharmacists to manage medicine inventory. Hospital administrators monitor billing, staff, and overall operations. Analyze the system and explain how object-oriented principles (abstraction, encapsulation, inheritance, and polymorphism) can be applied to model the system. Identify the major classes and relationships, and recommend a suitable SDLC model with justification.

1. UNDERSTANDING THE CASE

The hospital brings together five distinct kinds of participants — Patient, Doctor, Laboratory staff, Pharmacist, and Administrator — each with its own responsibility (booking, diagnosis, testing, dispensing, oversight) but all needing to work against a shared pool of patient data. The central design challenge is therefore twofold: model five different actor roles without duplicating their common behaviour (login, profile management), and keep sensitive clinical data — medical history, test results, prescriptions — properly protected while still letting the right roles read and update it. This is exactly the kind of requirement object-oriented analysis is suited for.

2. APPLICATION OF OBJECT-ORIENTED PRINCIPLES

- **Abstraction —** the system exposes only what each role needs through simple operations such as bookAppointment(), addDiagnosis(), or uploadReport(), while hiding how appointments are scheduled internally, how records are persisted, or how conflicts are resolved. A doctor calling record.addDiagnosis() does not need to know how the record is stored or indexed — the complexity is abstracted behind the method.
- **Encapsulation —** each class keeps its own data private and controls access through defined methods. A Patient's medical history, for instance, is not a public field that any class can edit directly; it is accessed only through MedicalRecord's own methods, so a Pharmacist object cannot accidentally (or maliciously) alter a diagnosis. This is critical in healthcare, where uncontrolled access to patient data is both an ethical and a compliance risk.
- **Inheritance —** Patient, Doctor, LabTechnician, Pharmacist, and Administrator all share common attributes (userId, name, contactNumber) and behaviour (login(), viewProfile()). Rather than repeating these in every class, they are pulled up into a common abstract User class that each role inherits from, so shared logic is written once and specialised behaviour is added only where it differs.

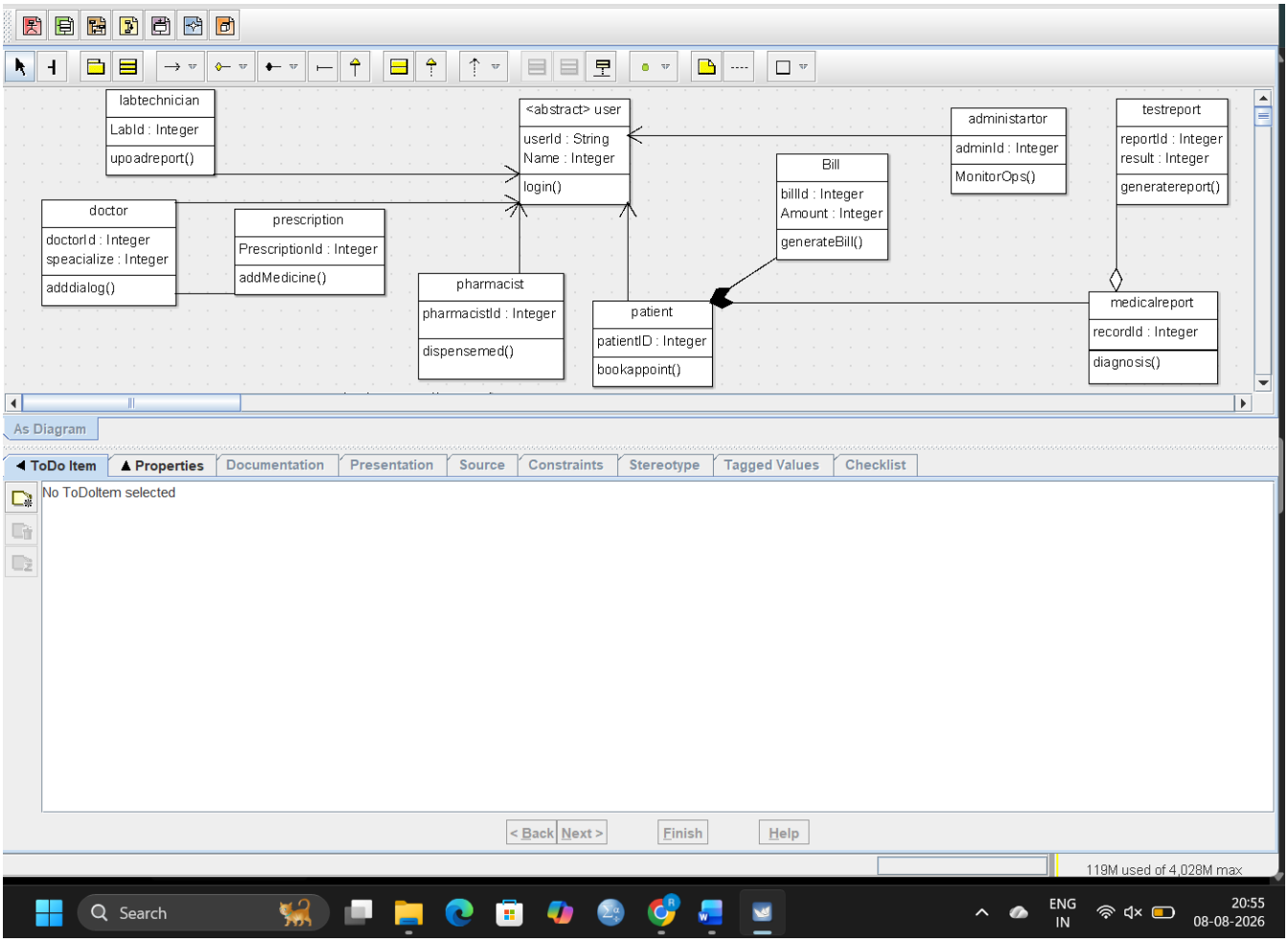
- **Polymorphism —** a single call such as user.viewDashboard() behaves differently depending on which subclass the object actually is at runtime — a Doctor sees today's appointments and pending diagnoses, a Pharmacist sees low-stock alerts, and an Administrator sees billing summaries. The rest of the system can treat every logged-in user uniformly through the User interface without needing to know or check which specific role it is handling, and new roles can be added later (e.g. a Nurse) without changing any existing calling code.

3. MAJOR CLASSES AND RELATIONSHIPS

Class	Description
User (abstract)	Base class for every person who logs into the system; holds shared attributes and login()/viewProfile().
Patient	Specialises User; books appointments and views their own medical history and bills.
Doctor	Specialises User; attends appointments and maintains medical records and prescriptions.
LabTechnician	Specialises User; uploads test reports linked to a patient's medical record.
Pharmacist	Specialises User; manages the medicine Inventory and fulfils prescriptions.
Administrator	Specialises User; oversees Billing, staff records, and overall hospital operations.
Appointment	Links a Patient to a Doctor at a scheduled time; the starting point of a clinical encounter.
MedicalRecord	Belongs to exactly one Patient; accumulates diagnoses, prescriptions, and test reports over time.
TestReport	Uploaded by a LabTechnician and attached to a patient's MedicalRecord.
Prescription	Written by a Doctor against a MedicalRecord; references one or more Medicine items.
Inventory / Medicine	Tracks stock levels of medicines managed by the Pharmacist.
Bill	Generated per patient encounter and monitored by the Administrator.

Key relationships: Patient and Doctor →→ Appointment (association, many-to-many over time, 1 to 0..*); Patient ◆— MedicalRecord (composition, 1 to 1, since a record has no meaning outside its owning

patient); MedicalRecord ◇— TestReport (aggregation, 1 to 0..*, since reports are gathered into a record but originate from the lab); Doctor — Prescription (association, 1 to 0..*); Prescription — Medicine (association, 0..* to 0..*); Pharmacist — Inventory (association, 1 to 1); Patient ◆— Bill (composition, 1 to 0..*).



4. RECOMMENDED SDLC MODEL

The Incremental Model is the most suitable choice for this system. The hospital's requirements naturally decompose into largely independent modules — appointment booking, electronic medical records, lab report management, pharmacy inventory, and billing/administration — each of which can be designed, built, and deployed as a separate increment rather than waiting for the entire system to be finished at once.

Justification:

- Modules map cleanly onto increments: appointment booking and basic patient records can go live first (delivering immediate value to staff and patients), while lab integration, pharmacy inventory, and administrative billing are added in subsequent increments.

- Lower risk: because each increment is a smaller, working subsystem, defects and requirement misunderstandings surface early and are cheaper to fix than if discovered only at the end of a single long development cycle.
- Stakeholder feedback: hospital staff can start using early increments and provide feedback that shapes later increments, which matters here because doctors, lab technicians, and administrators each have workflow habits that are hard to fully specify up front.
- Regulatory and safety considerations in healthcare software benefit from the more predictable, phase-gated nature of each increment compared to a purely continuous Agile flow, while still retaining the flexibility to adjust later increments based on real usage.

Case 2. Smart Banking System [20 Marks]

A national bank plans to develop a Smart Banking System that supports account creation, fund transfers, online bill payments, loan processing, and transaction monitoring. Customers, bank employees, loan officers, and administrators interact with the system through web and mobile applications. Design the system using object-oriented concepts by identifying key classes, objects, and relationships. Explain how object-oriented design improves software quality and recommend the most appropriate SDLC model for developing this banking application with suitable justification.

1. UNDERSTANDING THE CASE

A banking platform has to satisfy two requirements that pull in different directions: it must support a growing, changing set of features (fund transfers, bill payments, loans, and — implicitly — future services like UPI or digital wallets), while at the same time guaranteeing that money-related operations are accurate, secure, and auditable at every step. Object-oriented design is well suited to this tension because it lets new transaction and account types be added later without touching the code that already works, while encapsulation keeps sensitive financial data from being modified through anything other than controlled, validated operations.

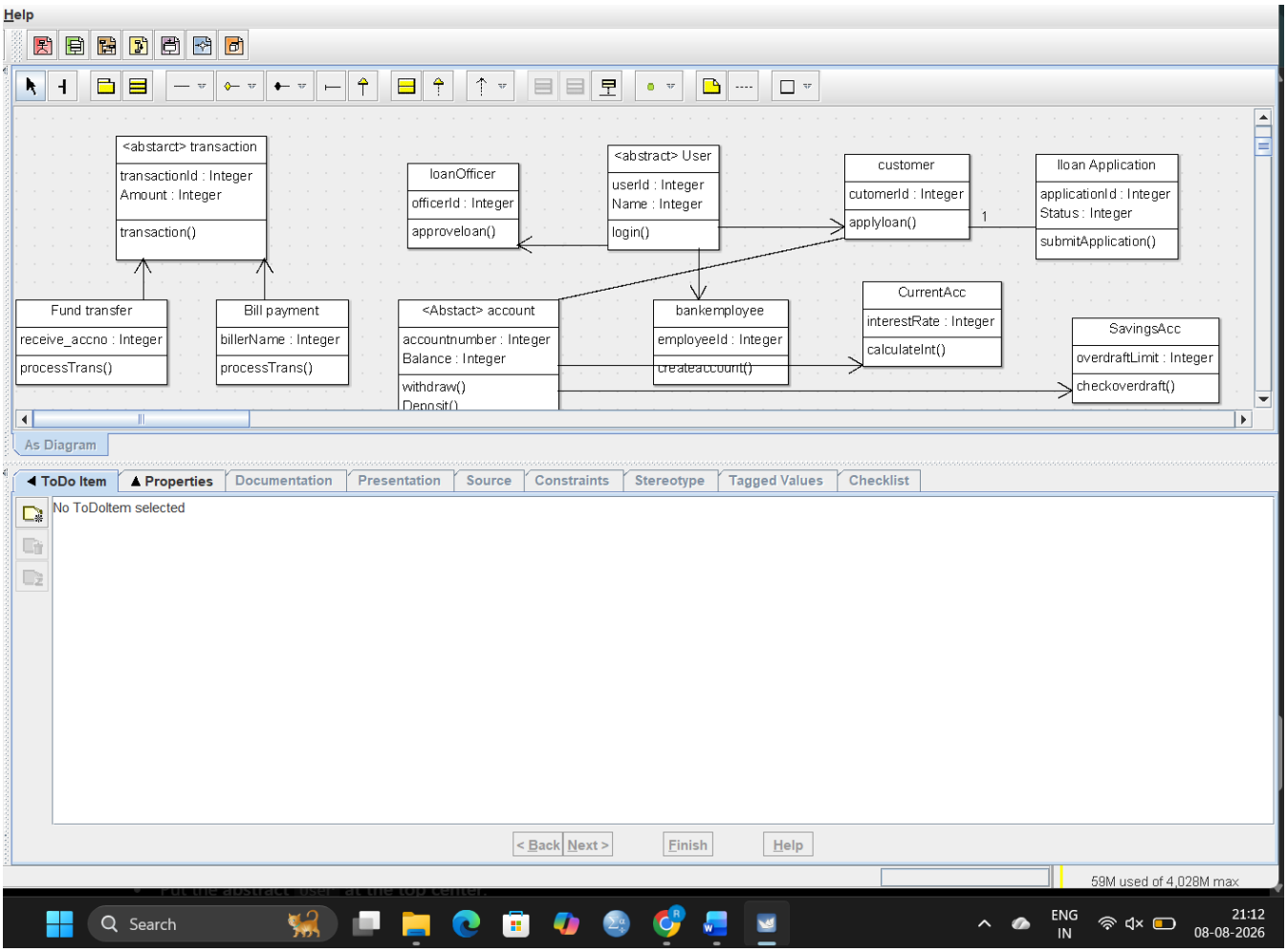
2. KEY CLASSES, OBJECTS, AND RELATIONSHIPS

Class	Description
User (abstract)	Base class for anyone who logs in; holds shared login credentials and profile behaviour.
Customer	Specialises User; owns one or more Accounts and initiates transactions and loan applications.
BankEmployee	Specialises User; creates and manages customer accounts.
LoanOfficer	Specialises User; reviews and approves/rejects LoanApplication objects.
Administrator	Specialises User; oversees system-wide user management and monitoring.
Account (abstract)	Base class holding accountNumber and balance, with deposit()/withdraw()/checkBalance().
SavingsAccount / CurrentAccount	Specialise Account; add interest calculation or overdraft rules respectively.

Class	Description
Transaction (abstract)	Base class representing any movement of money, with an overridden processTransaction().
FundTransfer / BillPayment	Specialise Transaction; each implements processTransaction() with its own validation and settlement logic.
LoanApplication / Loan	A Customer applies for a LoanApplication; once approved by a LoanOfficer it becomes an active Loan.

An object-level example makes this concrete: a specific object such as aliceAccount : SavingsAccount with balance = 42,500 is one instance of the Account hierarchy at a moment in time, and a corresponding txn217 : FundTransfer object records one particular transfer out of that account — the class diagram defines the blueprint, while such objects are the runtime data the bank actually operates on.

Key relationships: Customer — Account (association, 1 to 1..*, a customer typically holds at least one account); Account ◆— Transaction (composition, 1 to 0..*, since a transaction record has no meaning detached from the account it belongs to); Customer — LoanApplication (association, 1 to 0..*); LoanOfficer — LoanApplication (association, 1 to 0..*, an officer reviews many applications); LoanApplication — Loan (association, 1 to 0..1, an approved application produces exactly one active loan); BankEmployee — Account (association, 1 to 0..*, an employee creates/manages many accounts).



3. HOW OBJECT-ORIENTED DESIGN IMPROVES SOFTWARE QUALITY

- **Modularity —** each class owns a single, well-defined responsibility (Account manages balance, LoanApplication manages the approval workflow), so a change to how loans are processed does not risk breaking fund transfers.
- **Reusability —** common behaviour lives once in the User and Account base classes and is reused by every subclass, instead of being copy-pasted across Customer, BankEmployee, LoanOfficer, and Administrator, or across SavingsAccount and CurrentAccount.
- **Extensibility —** because Transaction and Account are designed with inheritance and polymorphism in mind, the bank can introduce a new transaction type such as UPIPayment or a new account type such as ForeignCurrencyAccount by adding a new subclass, without modifying or re-testing the existing, already-verified classes — this is the open/closed principle in practice.
- **Security and data integrity —** encapsulation ensures balance and PIN fields are never exposed directly; every change goes through validated methods such as withdraw(amount), which can

enforce business rules (sufficient balance, daily limits) consistently in one place rather than trusting every caller to apply them correctly.

- **Easier testing and maintenance —** because classes are loosely coupled through well-defined interfaces, individual classes such as FundTransfer can be unit-tested in isolation, and defects are easier to localise to a single class rather than spread across a monolithic procedure.

4. RECOMMENDED SDLC MODEL

The V-Model (Verification and Validation Model) is the most appropriate choice for this banking application. Unlike the hospital case, where independent modules and evolving staff workflows favoured an incremental rollout, a banking system's overriding concern is correctness and security of every transaction before release, since defects here have direct financial and regulatory consequences.

Justification:

- Traceability between requirements and testing: the V-Model pairs every development phase (requirements, design, coding) with a corresponding testing phase (acceptance testing, system testing, unit testing) planned in parallel, so test cases for fund transfers, loan processing, and transaction monitoring are defined from the requirements stage itself rather than as an afterthought.
- High reliability requirement: banking regulations and customer trust demand rigorous verification before go-live; the V-Model's phase-gated validation is better suited to this than a purely exploratory Agile flow for the core transactional engine.
- Well-understood domain: core banking operations (accounts, transfers, payments) are well-established and unlikely to change drastically mid-project, which suits the V-Model's more structured, less iterative nature better than it would suit a system with highly uncertain or fast-changing requirements.
- Note: for the customer-facing web/mobile application layers, where UI and feature requirements are more likely to evolve based on user feedback, the bank may supplement the V-Model with Agile sprints — but for the core account/transaction/loan engine described in this case, the V-Model's rigour is the stronger fit.